

SynthGraph — Design

Document & Specification

A functional, text-first language and build system for creating audio samples and full songs through composable, pure mathematical transformations on sound.

Status: Draft (design phase)

Platform: Linux desktop

Implementation language: Modern C++

Document type: Living design reference — captures decisions *and* the reasoning behind them.

Table of Contents

1. [Vision & Philosophy](#)
2. [Core Concepts & Terminology](#)
3. [The Language](#)
4. [The Type System](#)
5. [Core Primitives & Semantics](#)
6. [The Primitive Library](#)
7. [Modules & Code Organization](#)

8. [Projects & the Build System](#)
 9. [The Dev App](#)
 10. [Architecture & Separation of Concerns](#)
 11. [Technology Stack](#)
 12. [Milestones — Epics, Issues & Dependencies](#)
 13. [Future Work & Open Questions](#)
 14. [Glossary](#)
-

1. Vision & Philosophy

SynthGraph is a **programming language for sound**. A sound — from a short sample up to a fully arranged song — is written as an ordinary source file: a composition of pure functions that generate, transform, slice, and arrange signals. The project is compiled by a build system, and the resulting audio artifacts are browsed and played in a companion dev app.

SynthGraph deliberately ships **no editor**. The product surface is a compiler, a linter, and a build daemon; any text editor or IDE is the authoring environment. This keeps the implementation small and lets the entire ecosystem of editing, diffing, and version control (git) apply to audio projects for free.

Guiding tenets

- **Sound is code.** A song is a program: readable, diffable, refactorable, version-controlled. Reuse is function definition; arrangement is function composition.
- **Purity by construction.** The language is purely functional. There are no side effects anywhere in user code except one: the `render` primitive, which writes an audio artifact. Everything else is a pure value computation, which is what makes caching, incremental rebuilds, and parallel evaluation tractable.
- **Source-agnostic pipeline.** A synthesized generator and an imported audio file produce the same type of value. Nothing downstream cares about a signal's provenance.
- **Correctness is compiled.** A project is type-checked before it renders. Invalid programs fail at build time with diagnostics against source locations, not silently at runtime.
- **Small language, real library.** The language itself is deliberately minimal — a handful of types, functions, tuples, and lists. Expressive power comes from the

primitive library and from composition, not from language features.

- **Editor-agnostic tooling.** Compiler + linter + daemon + dev app. The build system owns caching and incremental recomputation internally; the user never tunes it.

2. Core Concepts & Terminology

Term	Meaning
Signal	An <i>infinite</i> function of time yielding sound values. Conceptually continuous; discretization happens only at render time.
Sample	A <i>finite</i> , timed slice of a Signal, with a start and end timestamp. The unit of rendering and arrangement.
Timestamp	A point in time measured since the epoch — the beginning ($t = 0$) of the signal timeline. Written with unit suffixes (<code>100ns</code> , <code>30us</code> , <code>10ms</code> , <code>1s</code> , <code>1m</code>).
Scalar	A single numeric value (a pitch, a decay rate, an amplitude).

Term	Meaning
Vector	A list of Scalars. As a Signal element type, it represents an N-channel frame.
Render target	A named artifact declared in source via the <code>render</code> primitive; the unit the build system produces and the dev app plays.
Project	A directory of <code>.synth</code> source files governed by a <code>.build</code> manifest.
Build daemon	A continuously running background process that watches the project and (re)compiles it.
Dev app	A desktop application that reads build metadata and lets the user browse and play the project's rendered artifacts.

The historically overloaded word "sample" is fixed by the type system: a **Sample** is always the finite, timed slice type. An individual measurement inside a discretized buffer is a *frame value* and only exists inside the engine.

3. The Language

The syntax is **OCaml-like**: `let` bindings, function definitions with named parameters, `val` signatures for primitives, expression-oriented style, definitions terminated with `;;`.

3.1 Design constraints

- **Minimal by intent.** Variable definitions, functions with multiple parameters, tuples, lists, and (limited) higher-order functions. No pattern matching, no branching, no user-defined types, no recursion in v1.
- **No type inference.** Every function parameter and every return type is explicitly annotated. The checker verifies; it never guesses. This keeps both the implementation and the reading experience simple.
- **Pure, except `render`.** `unit` is the effect type, and `render` is the only primitive that produces it. A `let _ = render ...` at top level is how a build target is declared.
- **No Booleans, no control flow (v1).** There is no `Bool` type, no `if`, no `match`.

Programs are pure expression composition.
Conditional logic is future work.

3.2 Function definition

```
let f amp:Scalar freq:Scalar : Scalar  
Signal =  
  (sine freq) * amp  
;;
```

- Parameters are annotated `name:Type` .
- The return type follows the parameter list, after `:`.
- The body is a single expression.

3.3 Higher-order functions (limited)

Function types may appear in signatures, and any user-defined function whose signature matches may be passed as an argument:

```
val map : f:( 'a -> 'b) -> xs:'a list -> 'b  
list
```

The intent is pragmatic: enough higher-order capability to make list processing (`map`, `fold`) expressible with user-supplied functions, while keeping the language simple in spirit. *(Low-confidence detail: whether anonymous functions/lambdas are*

*included in v1 or only named functions may be passed
– leaning named-only for
simplicity; to be validated during implementation.)*

3.4 A complete example

```
(* pluck.synth *)

let pluck freq:Scalar : Scalar Signal =
  (sine freq) * (exp_decay 6.0)
;;

let pluck_sample freq:Scalar : Scalar
Sample =
  sample (pluck freq) 0s 800ms
;;

let place_pluck at:Timestamp : Scalar
Signal =
  place (pluck_sample 440.0) at
;;

let song : Scalar Signal =
  mix_all (map place_pluck [0s; 500ms; 1s;
1500ms])
;;

let _ = render "demo" 48000.0 (sample song
0s 2s)
;;
```


Building this project produces one artifact named `demo` : two seconds of audio containing four decaying 440 Hz plucks.

4. The Type System

4.1 Primitive types

Type	Description
<code>Scalar</code>	A single numeric value.
<code>Vector</code>	A list of Scalars; the N-channel frame type.
<code>Timestamp</code>	Time since epoch; literals carry unit suffixes (<code>ns</code> , <code>us</code> , <code>ms</code> , <code>s</code> , <code>m</code>). All suffixes denote the same underlying quantity.
<code>String</code>	Text; used for artifact names and file paths.
<code>unit</code>	The effect type; produced only by <code>render</code> .

4.2 Parameterized types

`Signal` and `Sample` are **parameterized by their element type** (phantom-type style):

Type	Meaning
<code>Scalar Signal</code>	A mono signal — one value per instant.
<code>Vector Signal</code>	An N-channel signal — a Vector of channel values per instant.
<code>Scalar Sample</code> <code>/ Vector Sample</code>	Finite slices of the corresponding signal kind.

The element parameter **propagates** through the whole slicing/arrangement layer:

`sample`, `place`, and `render` are polymorphic in it (`'a Signal`, `'a Sample`), so the arrangement machinery is channel-agnostic.

`list` is a **general type former**, usable anywhere:

`Timestamp list`,

`Scalar Sample list`, `Scalar Signal list`, etc.

`Vector` is conceptually a

`Scalar list` specialized for the channel-frame role.

Tuples exist for multiple return values:

```
let analyze s:Scalar Signal : (Scalar
Signal, Scalar) = ... ;;
```

4.3 Polymorphism boundary

User code is fully annotated and monomorphic. Type variables ('a) appear only in the signatures of **built-in primitives** (e.g. `map` , `sample` , `place` , filters). The checker instantiates them at call sites; users cannot declare new polymorphic functions in v1. *(Low-confidence: this restriction is the simplest consistent reading of "no inference + limited higher-order"; revisit if it proves too tight.)*

4.4 Operators

Arithmetic operators are **lifted pointwise** onto Signals and Vectors:

- `a + b` on two Signals mixes them; `a * env` applies an envelope; etc.
- Scalars broadcast: `s * 0.5` attenuates a signal by half.
- Vector arithmetic is element-wise; both operands must have the same channel count.

(Open question: channel count is generally not static, so mismatch handling —

build-time where provable, otherwise a defined runtime rule — needs a decision.)

5. Core Primitives & Semantics

5.1 Signal and Sample

A **Signal is infinite**: it is defined for every `Timestamp` $\geq 0s$ (the epoch). A

Sample is finite: a slice of a signal between two timestamps.

```
val sample : signal:'a Signal ->
  from:Timestamp -> to:Timestamp -> 'a Sample
val place  : sample:'a Sample ->
  at:Timestamp -> 'a Signal
```

- `sample` cuts the window `[from, to)` out of a signal.
- `place` embeds a Sample back into the infinite time domain starting at `at`; the resulting Signal is **silence everywhere outside the placed window**.

Arrangement therefore needs no dedicated timeline concept: a song is an ordinary expression — Samples placed at Timestamps and mixed:

```
mix_all [place kick 0s; place kick 500ms;
place melody 250ms]
```

Repetition, echoes, and patterns are list construction plus `place` plus mixing.

5.2 Render — the single effect

```
val render : name:String -> rate:Scalar ->
sample:'a Sample -> unit
```

- `render` is the **only side-effecting primitive** in the language. It declares a build target: the given `Sample`, discretized at `rate` (Hz) and written as an audio artifact identified by `name`.
- Discretization is exclusively a render-time concern. Signals are conceptually continuous; sample rate never appears anywhere else in the language.
- **Duplicate render names anywhere in the project are a build error.** Names are the stable identifiers by which the build system and the dev app refer to artifacts.
- Idiom: `let _ = render "kick-test" 48000.0 (sample kick_sig 0s 1s) ;;`. Wrapping a work-in-progress expression in a temporary `render` is also the standard way to

listen to an intermediate result during development.

5.3 File import

```
val load_mono  : path:String -> Scalar  
Signal  
val load_multi : path:String -> Vector  
Signal
```

- Imported audio files are legitimate signal sources, indistinguishable downstream from synthesized ones.
- The file's actual channel count is checked **at build time** against the primitive used: loading a stereo file with `load_mono` (or vice versa) is a build error. The compiler can perform this check because audio files are **build inputs** — the daemon watches them like source files.
- A loaded file occupies `[0s, duration)` of the resulting Signal and is silence afterward (Signals are infinite).

6. The Primitive Library

The language is small; the library is where the DSP lives. The v1 set:

Generators

- `sine`, `saw`, `square` — basic oscillators
(`freq:Scalar -> Scalar Signal`).

Envelopes

- `adsr` — attack/decay/sustain/release envelope, produced as a *Signal* and applied by multiplication.
- `exp_decay` — exponential decay envelope.

Filters

- `lowpass`, `highpass` — `cutoff:Scalar -> 'a Signal -> 'a Signal`.

Arithmetic & combination

- Pointwise `+`, `-`, `*`, `/` on Signals and Vectors, with Scalar broadcasting.
- `mix_all` : `'a Signal list -> 'a Signal` — n-ary mix.
- `channels` : `Scalar Signal list -> Vector Signal` — combine mono signals into one N-channel signal (e.g. `channels [left; right]` for stereo).

Slicing, arrangement, I/O

- `sample`, `place`, `render`, `load_mono`,
`load_multi` (§5).

List combinators

- `map`, `fold` — the higher-order workhorses for building note lists, placing patterns, and folding mixes.

(Low-confidence details: exact primitive signatures — in particular whether `adsr` parameters are Timestamps or Scalars — and the full combinator roster beyond `map/fold/mix_all`. A feedforward `delay : Timestamp -> 'a Signal -> 'a Signal` convenience primitive is under consideration; `place` already covers the same ground at the Sample level.)

7. Modules & Code Organization

A file is a module. Every `.synth` file's top-level definitions form a module named after the file.

```
(* a.synth *)
let f amp:Scalar freq:Scalar : Scalar
Signal = ... ;;
```



```
(* b.synth *)  
import A  
  
let g : Scalar Signal = A.f 0.8 440.0 ;;
```

- `import A` resolves `a.synth` **in the same directory** (v1 rule).
 - All top-level definitions are visible to importers in v1.
 - This is the extensibility story: reusable instrument and effect libraries are ordinary `.synth` files, shared as source.
 - Deferred, deliberately: interface files / export lists (visibility control), library packaging, cross-directory or dependency resolution.
-

8. Projects & the Build System

8.1 The project manifest

A project directory contains a `.build` file that instructs the build system.

v1 contents:

- **Project name.**
- **The list of** `.synth` **source files** that constitute the project.

Everything else (output directories, dependencies, per-project defaults) is deferred.

8.2 What a build does

1. **Parse & type-check** every source file (all diagnostics carry source spans).
2. **Validate project-level rules** — e.g. duplicate `render` names, unresolved imports, channel-count mismatches on `load_*` (audio files are build inputs).
3. **Enumerate render targets** — every `render` call is a target.
4. **Evaluate & discretize** each target at its declared rate and write the audio artifact.
5. **Emit build metadata** — the machine-readable index of the project's targets and artifact locations that the dev app consumes.

8.3 The build daemon

The build system runs as a **continuously running background daemon** that watches the project (sources, `.build`, imported audio files) and rebuilds on change. The editing loop is: save file → daemon rebuilds → dev app reflects the new artifacts.

A one-shot CLI build (same engine, no watching) exists for scripting and CI-style use. The **linter** runs the same front-end checks standalone, for editor integration.

8.4 Caching & incremental recomputation — internal and automatic

Because the language is pure, every expression's value is a deterministic function of its inputs. The daemon exploits this:

- **Caching is fully automatic** at the build-system level. Which intermediate values are memoized, and when they are evicted, is an internal engine concern driven by heuristics — there are **no user-facing cache controls**.
- **Incremental rebuilds**: an edit invalidates only the targets (and cached intermediates) that depend on the changed definitions; everything else is reused.
- **Parallelism**: with a purely acyclic dependency structure (no feedback in v1), independent targets and independent subexpressions parallelize naturally across a thread pool.

Incrementality and parallelism are architectural commitments, but **not MVP**

blockers: a naive whole-project recompiler is an acceptable v0 of the daemon (§12).

9. The Dev App

A single desktop application for interacting with build output. Scope is deliberately narrow:

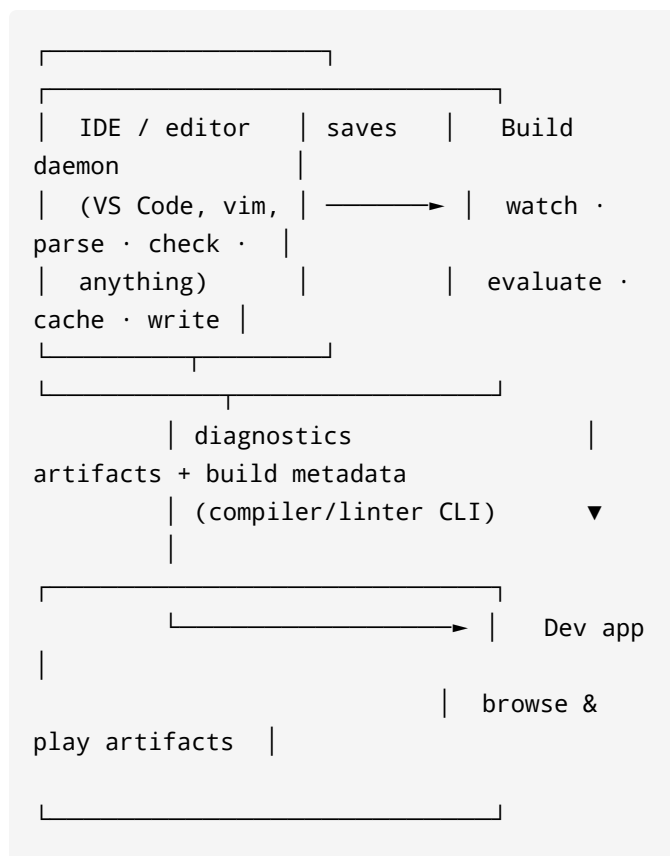
- Reads the **build metadata** of a project and lists all **render targets** by name.
- **Plays** any rendered artifact; shows basic facts (duration, rate, channels, build freshness/errors).
- Live-updates as the daemon rebuilds.

What it is **not** (v1): not an editor, not a profiler, not a mid-expression

inspector. **Only rendered data is auditionable.** The workflow for listening to an intermediate value is to wrap it in a temporary, named `render` — a cheap, git-visible idiom that needs no extra machinery.

10. Architecture & Separation of Concerns

Three cooperating processes, plus whatever editor the user prefers:



- The **compiler/linter** is a plain CLI: editors integrate with it for diagnostics

however they like; no bespoke editor tooling is shipped in v1.

- The **daemon** owns all computation: the language front-end, evaluation, caching, incremental rebuilds, artifact writing, metadata emission.
- The **dev app** is a pure consumer of build outputs; it never talks to the compiler internals. (*Low-confidence detail: whether the app tails the metadata file or the daemon exposes a small local IPC/socket for change notification — file-watching the metadata is the simplest v1.*)

This separation keeps every component replaceable: the daemon has no UI, the dev app has no language knowledge, and the editor is not ours to build.

11. Technology Stack

Concern	Choice	Notes
Implementation language	Modern C++	Compiler, daemon, dev app.

Concern	Choice	Notes
Product surface	Compiler + linter CLI, build daemon, dev app	No editor shipped.
Source language	SynthGraph DSL (<code>.synth</code>)	OCaml-like, pure, fully annotated.
Project manifest	<code>.build</code> file	Project name + source list (v1).
Persistence	Source files + <code>.build</code>	Source is the project format; human-readable, diffable, git-native.
Artifacts	Audio files (<code>.wav</code> first) + build metadata index	Metadata format is machine-readable (likely JSON).
Platform	Linux desktop	Primary (and initially only) target.

Concern	Choice	Notes
Dev app UI substrate	SDL2 (+ Dear ImGui)	Suitable for a small player/browser tool. (<i>Low-confidence; any lightweight toolkit works – decide at Epic 8.</i>)

12. Milestones — Epics, Issues & Dependencies

Timelines are expressed as issues and dependency ordering, not calendar dates — the project is worked on intermittently. Arrows (→) denote hard dependencies.

MVP definition

A project can be compiled and introspected: write

`.synth` files + `.build`,
run the build, and browse/play the resulting artifacts in the dev app. A naive whole-project rebuild is acceptable; the MVP proves the language, the build

pipeline, and the artifact/metadata contract.
Incrementality and parallelism are
fast-follows, not MVP gates.

Epic 0 — Foundations

- [] **0.1** Repo structure, build system, C++ toolchain conventions.
- [] **0.2** Language specification document (grammar, type rules) as the implementation reference.

Epic 1 — Language Front-End

(depends on 0.2)

- [] **1.1** Lexer (incl. Timestamp unit-suffix literals) and parser → AST.
- [] **1.2** Source-span tracking on every AST node (diagnostics currency).
- [] **1.3** Module resolution: `import` , same-directory rule, file = module.
- [] **1.4** Parse-error diagnostics with spans.

Epic 2 — Type Checker

(depends on 1.1)

- [] **2.1** Primitive types (Scalar, Vector, Timestamp, String, unit), tuples, general `list` .

- [] **2.2** Parameterized `'a Signal` / `'a Sample` with element-type propagation.
- [] **2.3** Fully-annotated checking (no inference); polymorphic primitive instantiation at call sites.
- [] **2.4** Function types in signatures; higher-order argument checking.
- [] **2.5** Operator typing (pointwise lifting, Scalar broadcasting).
- [] **2.6** Type-error diagnostics with spans.

Epic 3 — Evaluator & Signal Engine

(depends on 2.x)

- [] **3.1** Internal signal representation and buffer model (discretization machinery, block processing).
- [] **3.2** Evaluation of pure expressions to signal/sample values.
- [] **3.3** `sample` / `place` semantics (windowing, silence outside windows).
- [] **3.4** Render-time discretization at target rate; `.wav` writing.

Epic 4 — Primitive Library

(depends on 3.1)

- [] **4.1** Generators: sine, saw, square.

- [] **4.2** Envelopes: ADSR, exponential decay.
- [] **4.3** Filters: lowpass, highpass.
- [] **4.4** Arithmetic operators on Signals/Vectors + broadcasting.
- [] **4.5** `channels`, `mix_all`, `map`, `fold`.
- [] **4.6** `load_mono` / `load_multi` with build-time channel validation.

Epic 5 — Build System (one-shot)

(depends on 1.x, 2.x, 3.4)

- [] **5.1** `.build` manifest parsing (project name, source list).
- [] **5.2** Project-level validation: duplicate render names, unresolved imports.
- [] **5.3** Target enumeration from `render` calls; artifact output layout.
- [] **5.4** Build metadata emission (target index for the dev app).
- [] **5.5** CLI: one-shot build + standalone linter mode.

Epic 6 — Build Daemon

(depends on 5.x)

- [] **6.1** Watch mode: sources, `.build`, imported audio files.

- [] **6.2** Rebuild-on-change loop (whole-project rebuild acceptable here).
- [] **6.3** Robust error surfacing in metadata (partial-failure builds).

Epic 7 — Dev App

(depends on 5.4)

- [] **7.1** Project open → target list from build metadata.
- [] **7.2** Artifact playback + basic facts (duration, rate, channels, status).
- [] **7.3** Live refresh on metadata change.

★ **MVP boundary:** Epics 0–7.

Epic 8 — Automatic Caching & Incremental Builds

(depends on 6.x)

- [] **8.1** Dependency tracking across definitions/modules/targets.
- [] **8.2** Automatic memoization of intermediate values (internal heuristics).
- [] **8.3** Invalidation: rebuild only affected targets on edit.
- [] **8.4** Cache memory budgeting/eviction.

Epic 9 — Parallel Evaluation

(depends on 8.1)

- [] 9.1 Parallel builds across independent targets.
 - [] 9.2 Thread-pool scheduling of independent subexpressions.
-

13. Future Work & Open Questions

Explicitly **out of v1 scope**, recorded so the door stays visibly open:

- **Feedback & cycles.** Self-referential signals (IIR-style feedback, Karplus-Strong, feedback delays) are **not supported in v1**. The language is purely acyclic composition. Adding feedback later means recursive signal bindings plus a delay-legality rule and a sequential evaluation mode — a substantial, deliberate future extension.
- **Booleans & control flow.** No `Bool`, `if`, or `match` in v1; conditional structure is future work.
- **Visibility & libraries.** Interface files / export lists, packaging, and dependency resolution beyond same-directory imports.

- **Cache tuning.** Caching stays automatic; user-facing controls only if heuristics prove insufficient.
 - **Observability.** No tracing/profiling instrumentation in v1. If deep introspection (timings, cache analytics) returns later, it is a retrofit with known cost.
 - **Native extensions.** User-authored compiled (C++) primitives are out of scope; extensibility is source-level modules.
 - **Open questions.** Vector arithmetic channel-count mismatch semantics; lambda syntax vs named-functions-only for higher-order arguments; exact envelope/filter primitive signatures; a feedforward `delay` primitive; additional output formats and stem exports; dev-app change-notification mechanism (metadata watching vs IPC); multi-platform support (Linux-first for now).
-

14. Glossary

- **Signal** — An infinite function of time; `Scalar Signal` is mono, `Vector Signal` is N-channel.

- **Sample** — A finite slice of a Signal between two Timestamps; the unit of arrangement and rendering.
- **Timestamp** — Time since the epoch ($t = 0$ of the signal timeline); unit-suffix literals (`10ms` , `1s` , ...).
- **Epoch** — The beginning of a signal's timeline; the zero point all Timestamps measure from.
- **Render target** — A named artifact declared by a `render` call; unique names project-wide.
- **render** — The language's sole side-effecting primitive; discretizes a Sample at a given rate and writes it as an artifact.
- **.synth file** — A source file; also a module, importable by name.
- **.build file** — The project manifest (name + source list in v1).
- **Build metadata** — The machine-readable target/artifact index emitted by a build; the dev app's input.
- **Build daemon** — The background process that watches and rebuilds a project.
- **Dev app** — The artifact browser/player; the only shipped GUI.

- **Phantom element type** – The 'a' in 'a Signal' / 'a Sample'; carries channel structure (Scalar vs Vector) through slicing and rendering.